CXGate

class `qiskit.circuit.library.CXGate(*args, _force_mutable=False, **kwargs)`

GitHub ↗

Bases: `SingletonControlledGate`

Controlled-X gate.

Can be applied to a `QuantumCircuit` with the `cx()` and `cnot()` methods.

Circuit symbol:

```
q_0: ──■──
     ┌─┴─┐
q_1: ┤ X ├
     └───┘
```

Matrix representation:

$$CX\ q_0, q_1 = I \otimes |0\rangle\langle 0| + X \otimes |1\rangle\langle 1| = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

Note

In Qiskit's convention, higher qubit indices are more significant (little endian convention). In many textbooks, controlled gates are presented with the assumption of more significant qubits as control, which in our case would be q_1. Thus a textbook matrix for this gate will be:

```
     ┌───┐
q_0: ┤ X ├
     └─┬─┘
q_1: ──■──
```

$$CX\ q_1, q_0 = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

In the computational basis, this gate flips the target qubit if the control qubit is in the $|1\rangle$ state. In this sense it is similar to a classical XOR gate.

$$`|a, b\rangle \rightarrow |a, a \oplus b\rangle`$$

Create new CX gate.

Attributes

base_class

Get the base class of this instruction. This is guaranteed to be in the inheritance tree of `self`.

The "base class" of an instruction is the lowest class in its inheritance tree that the object should be considered entirely compatible with for _all_ circuit applications. This typically means that the subclass is defined purely to offer some sort of programmer convenience over the base class, and the base class is the "true" class for a behavioral perspective. In particular, you should *not* override `base_class` if you are defining a custom version of an instruction that will be implemented differently by hardware, such as an alternative measurement strategy, or a version of a parametrized gate with a particular set of parameters for the purposes of distinguishing it in a `Target` from the full parametrized gate.

This is often exactly equivalent to `type(obj)`, except in the case of singleton instances of standard-library instructions. These singleton instances are special subclasses of their base class, and this property will return that base. For example:

```
>>> isinstance(XGate(), XGate)
True
>>> type(XGate()) is XGate
False
>>> XGate().base_class is XGate
True
```

In general, you should not rely on the precise class of an instruction; within a given circuit, it is expected that `Instruction.name` should be a more suitable discriminator in most situations.

condition

The classical condition on the instruction.

Deprecated since version 1.3.0

The property `qiskit.circuit.instruction.Instruction.condition` is deprecated as of qiskit 1.3.0. It will be removed in 2.0.0.

condition_bits

Get Clbits in condition.

Deprecated since version 1.3.0

The property `qiskit.circuit.instruction.Instruction.condition_bits` is deprecated as of qiskit 1.3.0. It will be removed in 2.0.0.

ctrl_state

Return the control state of the gate as a decimal integer.

decompositions

Get the decompositions of the instruction from the SessionEquivalenceLibrary.

definition

Return definition in terms of other basic gates. If the gate has open controls, as determined from `ctrl_state`, the returned definition is conjugated with X without changing the internal `_definition`.

duration

Get the duration.

Deprecated since version 1.3.0

The property `qiskit.circuit.instruction.Instruction.duration` is deprecated as of qiskit 1.3.0. It will be removed in Qiskit 2.0.0.

label

Return instruction label

mutable

Is this instance is a mutable unique instance or not.

If this attribute is `False` the gate instance is a shared singleton and is not mutable.

name

Get name of gate. If the gate has open controls the gate name will become:

> <original_name_o<ctrl_state>

where <original_name> is the gate name for the default case of closed control qubits and <ctrl_state> is the integer value of the control state for the gate.

num_clbits

Return the number of clbits.

num_ctrl_qubits

Get number of control qubits.

Returns

The number of control qubits for the gate.

Return type

int ↗

num_qubits

Return the number of qubits.

params

Get parameters from base_gate.

Returns

List of gate parameters.

Return type

list ↗

Raises

CircuitError – Controlled gate does not define a base gate

unit

Get the time unit of duration.

Deprecated since version 1.3.0

The property `qiskit.circuit.instruction.Instruction.unit` is deprecated as of qiskit 1.3.0. It will be removed in Qiskit 2.0.0.

Methods

control

`control(num_ctrl_qubits=1, label=None, ctrl_state=None, annotated=False)`

GitHub ↗

Return a controlled-X gate with more control lines.

Parameters

- **num_ctrl_qubits** (*int* ↗) – number of control qubits.

- **label** (*str* ↗ / *None*) – An optional label for the gate [Default: `None`].
- **ctrl_state** (*str* ↗ /*int* ↗ / *None*) – control state expressed as integer, string (e.g. `'110'`), or `None`. If `None`, use all 1s.
- **annotated** (*bool* ↗) – indicates whether the controlled gate should be implemented as an annotated gate.

Returns

controlled version of this gate.

Return type

ControlledGate

inverse

`inverse(annotated=False)`

GitHub ↗

Return inverted CX gate (itself).

Parameters

annotated (*bool* ↗) – when set to `True`, this is typically used to return an `AnnotatedOperation` with an inverse modifier set instead of a concrete `Gate`. However, for this class this argument is ignored as this gate is self-inverse.

Returns

inverse gate (self-inverse).

Return type

CXGate